Here is a clean, exam-oriented **Introduction to the .NET Framework** in simple Indian-English style:

Introduction to the .NET Framework

The **.NET Framework** is a software development platform created by **Microsoft**. It provides a **runtime environment**, **class libraries**, and **tools** for building and running applications on Windows.

It supports multiple programming languages (like **C#, VB.NET, F#**) and allows them to work together smoothly.

Key Components of .NET Framework

1. CLR (Common Language Runtime)

- The **heart** of the .NET Framework.
- It manages the execution of .NET programs.
- Provides services like:
  - **Memory management** (Garbage Collection)
  - **Thread management**
  - **Exception handling**
  - **Security**
  - **JIT compilation** (Just-In-Time compilation)

2. BCL / FCL (Base Class Library / Framework Class Library)

- A large collection of **reusable classes and functions**.
- Contains libraries for:
  - Input/Output (File handling)
  - Networking
  - Database access (ADO.NET)
  - Collections

- XML processing
- Graphics (GDI+)

3. CTS (Common Type System)

- Defines how data types are declared and used.
- Ensures that all .NET languages follow a **common set of rules**.
- For example, `int` in C# == `Integer` in VB.NET.

4. CLS (Common Language Specification)

- A subset of CTS.
- Defines the **basic standards** that all .NET languages must support.
- Ensures **language interoperability**.

Architecture of .NET Framework

1. **Application Layer**
   - Applications written in C#, VB.NET, etc.
2. **Class Library Layer**
   - BCL/FCL
3. **CLR Layer**
   - Executes and manages applications.
4. **Operating System Layer**
   - Mainly Windows.

Features of .NET Framework

✔ Language Interoperability

Different languages can work together because of CTS and CLS.

✔ Memory Management

Garbage Collector removes unused objects automatically.

✔ Security

Built-in security features:

- Code Access Security
- Role-Based Security

✔ Cross-Language Debugging

Debug programs written in different languages.

✔ Large Framework Library

Reduces development time.

✔ Platform Support (Windows)

Primarily for building Windows apps.

Types of Applications Built with .NET Framework

- **Windows Forms applications**
- **ASP.NET Web applications**
- **WPF applications**
- **Console applications**
- **WCF services**
- **Web services**
- **Class libraries**

Execution Process in .NET

1. Code is written in C#, VB.NET, etc.
2. Compiler converts it into **Intermediate Language (IL)**.
3. CLR's **JIT Compiler** converts IL into **machine code**.
4. OS runs the program.

Advantages of .NET Framework

- Faster development due to large libraries
- High security
- Automatic memory handling

- Consistent programming model
- Supports multiple languages

Limitations

- Mainly Windows-based
- Heavy framework
- Less suitable for cross-platform development (solved in .NET Core / .NET 5+)

If you want, I can also provide:

✔ **Short notes version**
✔ **Diagram-based explanation**
✔ **Examples**
✔ **Difference between .NET Framework, .NET Core, and .NET 5+**

Just tell me!

Intermediate Language (IL)
Assemblies and their structure, EXEs/DLLs

Here are **clear, exam-oriented notes** on **Intermediate Language (IL)** and **Assemblies (EXE/DLL)** in a simple Indian-English style:

Intermediate Language (IL)

Intermediate Language (IL) is the **platform-independent** code generated by .NET compilers (like C# compiler).
It is also called:

- **MSIL** (Microsoft Intermediate Language)

- **CIL** (Common Intermediate Language)

The CLR converts IL into machine code using **Just-In-Time (JIT)** compilation at runtime.

Characteristics of IL

✔ Platform-Independent Code

IL is not tied to any specific CPU or operating system.

✔ High-Level + Low-Level Hybrid

It includes:

- High-level constructs → classes, methods, loops
- Low-level instructions → load, store, call

✔ Type-Safe Instructions

Ensures runtime safety.

✔ Verifiable Code

Before execution, CLR verifies IL for:

- Security
- Memory safety
- Valid type usage

IL Execution Steps

1. **Source Code → IL Code**
   (C# compiler produces IL + metadata)
2. **IL → Native Machine Code**
   (JIT compiler inside CLR)
3. **Native Code → Execution**
   (OS executes the code)

Example IL instruction:

arduino

Copy code

```
ldstr "Hello"
call void [mscorlib]System.Console::WriteLine(string)
```

.NET Assemblies

An **assembly** is the **basic unit of deployment, versioning, and security** in .NET.

It contains:

- IL code
- Metadata
- Manifest
- Resources

Assemblies are generated after compiling .NET code.

Types of Assemblies

1. EXE (Executable Assembly)

- Produces **executable programs**.
- Has an entry point:
  - `Main()` method (C#)
- Used to create applications like:
  - Console apps
  - Windows Forms apps
  - WPF apps

2. DLL (Dynamic Link Library Assembly)

- Does **not** have an entry point.
- Contains **reusable classes and functions**.
- Used in:
  - Class libraries
  - Web applications
  - Shared business logic

Structure of an Assembly

Every assembly has **four main parts**:

**1. Manifest

Contains details about the assembly, like:

- Assembly name
- Version number
- Culture information
- List of files
- Strong name (if signed)
- Dependencies (other assemblies)

This helps CLR load the assembly correctly.

**2. IL Code (CIL/MSIL)

Compiled intermediate language instructions for all methods.

**3. Metadata

Describes the types defined in the assembly:

- Classes
- Interfaces
- Methods
- Properties
- Namespaces

Metadata helps CLR with reflection, type-checking, and runtime binding.

**4. Resources

Optional data embedded into assembly:

- Images
- Strings
- Icons
- XML files

How Assemblies Work at Runtime

1. CLR loads the assembly.
2. CLR reads the **manifest** to understand:
   - Version
   - Dependencies
   - Culture
3. CLR verifies IL (security + type safety).
4. JIT compiler converts IL to machine code.
5. Application executes.

Private vs. Shared Assemblies

Private Assembly

- Stored locally in the application folder.
- Used by a single application.

Shared Assembly

- Stored in **GAC (Global Assembly Cache)**.
- Shared across multiple applications.
- Must have a **Strong Name**.

Strong-Named Assembly

Provides:

- Unique identity
- Versioning
- Security

Created using:

```nginx
sn -k keypair.snk
```

Copy code

Difference Between EXE and DLL

Feature	EXE	DLL
Purpose	Standalone application	Reusable library
Entry Point	Yes (Main)	No entry point
Runs Independently	Yes	No
Used By	End users	Other applications/assemblies
Deployment	One application	Many applications share it

If you want, I can also give:

- ✔ Diagram of assembly structure
- ✔ IL code example with explanation
- ✔ Short notes for exam
- ✔ Viva answers format

Feel free to ask!

CLR and its functions
□ JIT Compilation
□ Memory Management
□ Garbage Collection

- App Domain Management
- Memory Management
- CLS, CTS
- Security

Below are **clean, exam-oriented notes** on all CLR topics you mentioned — written in simple Indian-English, perfect for theory exams, viva, and interviews.

Common Language Runtime (CLR)

The **CLR** is the **execution engine** of the .NET Framework.
It is responsible for **running .NET applications**, providing services like:

- JIT Compilation
- Memory Management
- Garbage Collection
- App Domain Management
- Security
- Exception Handling
- Thread Management
- Type Safety

The CLR makes .NET applications **portable**, **secure**, **managed**, and **efficient**.

---

1. JIT Compilation (Just-In-Time Compilation)

After compilation, .NET code becomes **IL (Intermediate Language)**.
Before execution, CLR converts IL → **Native machine code** using a **JIT Compiler**.

Types of JIT Compilers

1. **Pre-JIT**
   - Compiles entire assembly to native code **at once**.
   - Used by **NGen (Native Image Generator)**.
2. **Econo-JIT**
   - Compiles only required methods.

- Saves memory by discarding unused methods.

3. **Normal JIT (Default)**
   - Compiles methods **just before execution**.
   - Stores compiled code in memory for future calls.

Advantages of JIT

- Platform independence (IL → Platform-specific code)
- Optimized performance
- Security through IL verification

---

2. Memory Management (Managed Memory)

In .NET, CLR takes care of memory allocation and freeing.

Programmers do not manually use `malloc()` or `free()`.

CLR Handles:

- Allocation of memory for objects
- Tracking object references
- Releasing unused memory using Garbage Collector

Memory is divided into:

- **Stack Memory –** value types, method calls
- **Heap Memory –** objects, reference types

---

3. Garbage Collection (GC)

Garbage Collector automatically frees memory that is **no longer needed**.

GC Working

1. Detects unreachable objects (not referenced by the program)
2. Frees their memory
3. Compacts the heap to remove memory gaps
4. Updates references

GC Generations

To optimise performance, objects are divided into:

- **Generation 0 –** short-lived objects (local variables)
- **Generation 1 –** medium-lived objects
- **Generation 2** – long-lived objects (static data, application-wide objects)

You can force GC manually using:

csharp Copy code

```
GC.Collect();
```

(but not recommended)

---

4. App Domain Management

Application Domains (AppDomains) isolate applications running inside a single process.

Purpose:

- Protection: One app cannot crash another
- Separate security boundaries
- Efficient resource usage
- Loading/unloading assemblies without restarting the whole process

Example:

- ASP.NET creates one AppDomain per application.

In .NET Core, AppDomains are replaced by **AssemblyLoadContext**.

---

5. CLS (Common Language Specification)

CLS defines a **set of rules** that all .NET languages must follow.
Ensures **language interoperability**.

Example:

- If library is written in C#, VB.NET must be able to use it.

CLS rules include:

- No unsigned types like `uint` in public APIs
- Case-insensitive naming in some languages
- Common exception rules

---

6. CTS (Common Type System)

CTS defines **how data types are declared, used, and managed**.

CTS Provides:

- Common data types across languages
- Rules for inheritance, polymorphism
- Rules for object lifetime and visibility

Example:

- `int` in C# `==` `System.Int32`
- `string` `==` `System.String`

CTs ensures **type safety** and **compatibility** between languages.

---

7. Security in CLR

CLR provides a **managed, safe execution environment**.

Types of Security:

1. Code Access Security (CAS)

Controls what code can do based on:

- Source (local/remote)
- Signature
- Publisher identity

For example:

- A file downloaded from the internet may not access system registry.

2. Role-Based Security (RBS)

Allows access based on user roles:

- Admin
- Guest
- Power user

3. Type Safety

Ensures correct memory usage and safe code execution.

4. IL Verification

Before execution, IL is verified for:

- Invalid pointers
- Stack underflow/overflow
- Type compatibility

5. Strong Names + GAC Security

Assemblies with strong names cannot be tampered with.

Short 2–3 Line Answers (Exam-Ready)

✔ What is CLR?

CLR is the execution environment of .NET that manages code execution, memory, JIT compilation, security, and garbage collection.

✔ What is JIT?

JIT converts IL code into native machine code at runtime just before execution.

✔ What is Garbage Collection?

GC automatically frees unused objects from memory to avoid memory leaks.

✔ What is CLS?

CLS defines a set of rules ensuring interoperability between .NET languages.

✔ What is CTS?

CTS defines how types are declared and used in .NET ensuring type safety and uniformity across languages.

✔ What is AppDomain?

AppDomains provide isolated environments within a process to run applications securely and independently.

If you want, I can prepare:

- ✔ **One-page notes**
- ✔ **Diagrams (CLR architecture, GC working)**
- ✔ **Viva questions and answers**
- ✔ **MCQs with answers**

Just tell me!

.Net Framework, .Net Core, Mono, Xamarin differences
Versions of the Framework

Here are **clear, crisp, exam-oriented notes** on **.NET Framework, .NET Core, Mono, Xamarin Differences** and **Versions of the .NET Framework** — written in simple Indian-English.

Differences: .NET Framework vs .NET Core vs Mono vs Xamarin

1. .NET Framework

- **Released:** 2002
- **Platform: Windows-only**
- **Type:** Full, traditional .NET
- **Use Case:**
  - Windows desktop apps (WinForms, WPF)

    - ASP.NET Web Forms / MVC (Windows)
    - Enterprise internal applications
- **Pros:** Mature, huge libraries, stable
- **Cons:** Not cross-platform, heavy, slower updates
- **Status:** Only maintenance mode (no major updates now)

---

2. .NET Core

- **Released:** 2016
- **Platform: Cross-platform** (Windows, Linux, macOS)
- **Type:** Modern, open-source successor
- **Use Case:**
    - High-performance web apps
    - Cloud, microservices
    - Command-line apps
- **Pros:**
    - Fast performance
    - Modular & lightweight
    - Cross-platform
    - Open-source
- **Cons:**
    - Does not support older .NET Framework features (Web Forms, WCF full)
- **Status:** Expanded into **.NET 5, 6, 7, 8 (Unified platform)**

---

3. Mono

- **Released:** Around 2004
- **Platform: Cross-platform** (Linux, macOS, Android, iOS, gaming consoles)
- **Type:** Open-source implementation of .NET Framework
- **Use Case:**
    - Linux/Unix desktop apps
    - Game development (Unity uses Mono)
- **Pros:**
    - Lightweight

    - Runs legacy .NET code on Linux
- **Cons:**
    - Limited Windows compatibility
    - Slower than .NET Core
- **Status:** Used internally inside Xamarin & Unity. Not primary future platform.

---

4. Xamarin

- **Released:** Acquired by Microsoft in 2016
- **Platform: Mobile development** (Android, iOS)
- **Type:** .NET + C# for mobile apps
- **Use Case:**
    - Cross-platform mobile apps using C#
- **Pros:**
    - Single codebase for Android+iOS
    - Native performance
- **Cons:**
    - UI code still written platform-specific (Xamarin.Android, Xamarin.iOS)
- **Status:** Merged into **.NET MAUI** starting .NET 6

Quick Comparison Table

Feature	.NET Framework	.NET Core	Mono	Xamarin
Platform	Windows only	Cross-platform	Cross-platform	Mobile platforms
Open-source	Partially	Fully	Fully	Fully
Performance	Medium	High	Medium	High (near-native)
Mobile Apps	No	No	Yes (via Mono)	Yes
Desktop Apps	Yes	Limited	Yes	No
Status	Legacy	Active	Used internally	Merged into MAUI

Versions of the .NET Framework

Here is a **clean list** of major .NET Framework versions with key features.

.NET Framework 1.0 (2002)

- First release
- Introduced CLR, IL, BCL, WinForms

.NET Framework 1.1 (2003)

- ASP.NET improvements
- ODBC support
- Mobile controls

.NET Framework 2.0 (2005)

- Major release
- Generics
- ASP.NET 2.0
- ClickOnce deployment

.NET Framework 3.0 (2006)

- Added 4 major technologies:
  - **WPF** (Windows Presentation Foundation)
  - **WCF** (Windows Communication Foundation)
  - **WF** (Workflow Foundation)
  - **CardSpace**

.NET Framework 3.5 (2007)

- **LINQ**
- **Entity Framework v1**
- ASP.NET AJAX

.NET Framework 4.0 (2010)

- Parallel programming (TPL)
- Improvements in CLR & GC

.NET Framework 4.5 (2012)

- Async/Await

- Better networking libraries
- ASP.NET Web API

.NET Framework 4.6 (2015)

- RyuJIT compiler
- High DPI support

.NET Framework 4.7 / 4.7.1 / 4.7.2 (2017–2018)

- Improved performance
- Better cryptography
- Touch/Ink support

.NET Framework 4.8 (2019)

- Last **major** version
- Better DPI
- Performance improvements
- Bug fixes

  No .NET Framework 4.9 or 5.0

  → Future is **.NET Core-based (.NET 5, 6, 7, 8)**

Easy Exam 2–3 Line Answers

✔ What is .NET Framework?

A Windows-only, mature development framework used for desktop and web apps. Includes CLR, BCL, ASP.NET, WinForms, WPF.

✔ What is .NET Core?

A modern, high-performance, cross-platform, open-source version of .NET designed for cloud and web applications.

✔ What is Mono?

An open-source implementation of .NET used for Linux, mobile devices, and Unity game development.

✔ What is Xamarin?

A mobile development framework using C# to build Android and iOS apps with near-native performance.

✔ Which .NET version is the latest?

Latest unified versions: **.NET 6 (LTS), .NET 7, .NET 8 (LTS)** — successor to .NET Core.

If you want, I can also give:

- ✔ Diagram of .NET ecosystem
- ✔ Difference between .NET 5/6/7/8
- ✔ Interview questions and MCQs
- ✔ Short notes PDFs

Just tell me!

Managedand UnmanagedCode
Introduction to Visual Studio
Using ILDASM

Here are **clear, simple, exam-oriented notes** on **Managed Code, Unmanaged Code, Visual Studio, and ILDASM** in Indian-English style.

---

1. Managed and Unmanaged Code

Managed Code

Managed Code is the code that runs **under the control of CLR (Common Language Runtime)**.

Characteristics

- CLR handles **memory management**, **Garbage Collection**, **security**, **exception handling**, etc.

- You do not manually allocate or free memory.
- Examples:
  - C#
  - VB.NET
  - F#

Advantages

- Automatic memory management
- Type safety
- High security
- No memory leaks

Examples

csharp Copy code

```csharp
// C# Managed Code
Console.WriteLine("Hello World");
```

Unmanaged Code

Unmanaged Code is the code **NOT managed by CLR**.

Memory is manually handled by the programmer.

Characteristics

- No garbage collection
- Uses OS APIs directly
- Unsafe memory operations
- Requires manual memory allocation like:
  - `malloc()`, `free()`

Examples

- C
- C++ (native code)
- COM Components
- Win32 APIs

Example

cpp

Copy code

```cpp
// C++ Unmanaged Code
printf("Hello World");
```

Key Differences

Feature	Managed Code	Unmanaged Code
Runs under	CLR	OS directly
Memory	Automatic (GC)	Manual
Safety	High	Low
Languages	C#, VB.NET	C, C++, Assembly
Errors	Less	More chances

---

2. Introduction to Visual Studio

Visual Studio is Microsoft's **Integrated Development Environment (IDE)** for developing .NET applications.

It supports:

- C#, VB.NET, F#, C++, Python, JavaScript
- .NET Framework, .NET Core, .NET 5/6/7
- ASP.NET Web Apps
- Windows Forms & WPF Apps
- Mobile Apps (Xamarin / MAUI)
- Cloud Apps (Azure)

Key Features of Visual Studio

✔ IntelliSense

Auto-suggestions for code: methods, variables, classes.

✔ Code Editor

Syntax highlighting, auto-completion, refactoring tools.

✔ Debugger

Step-by-step debugging: Breakpoints, watch windows, call stack.

✔ Designer Tools

Drag-and-drop designer for:

- WinForms
- WPF
- Web Forms

✔ NuGet Package Manager

Install third-party libraries easily.

✔ Integrated Git Support

Push, pull, merge, branches from inside Visual Studio.

✔ Extensions

Add-ons for:

- Themes
- Live Share
- Productivity tools

Benefits of Visual Studio

- All-in-one development environment
- Great debugging tools
- Easy UI designing
- Good productivity features
- Supports multiple languages and project types

---

3. Using ILDASM (IL Disassembler)

ILDASM (IL Disassembler) is a tool provided by Microsoft to view:

- IL Code
- Metadata
- Manifest
- Types and methods inside an assembly (EXE/DLL)

It helps you understand **how .NET code is compiled** into IL.

Purpose of ILDASM

- Inspect IL code of EXE/DLL
- View assembly manifest
- Debug low-level IL instructions
- Understand CIL (Common Intermediate Language)

How to Open ILDASM

Method 1: Using Developer Command Prompt

1. Open **Developer Command Prompt for Visual Studio**
2. Type:

nginx Copy code

```
ildasm
```

3. ILDASM window appears.

Method 2: Direct Path

Usually found in:

java Copy code

```
C:\Program Files (x86)\Microsoft SDKs\Windows\...\ildasm.exe
```

Using ILDASM

1. Open ILDASM
2. Go to **File → Open**
3. Select any `.exe` or `.dll` file
4. Expand the tree to see:
   - Manifest
   - Metadata
   - Classes
   - Methods
   - IL instructions

Example IL Code (Seen in ILDASM)

C# Code:

csharp Copy code

```csharp
Console.WriteLine("Hello");
```

ILDASM View:

arduino Copy code

```arduino
IL_0001: ldstr "Hello"
IL_0006: call void [System.Console]System.Console::WriteLine(string)
```

This helps developers understand what CLR executes internally.

Short Exam-Ready Answers

✔ What is Managed Code?

Code run inside CLR where memory management, security, and GC are handled automatically.

✔ What is Unmanaged Code?

Code that runs directly on OS without CLR support, requiring manual memory handling.

✔ What is Visual Studio?

A complete IDE from Microsoft used for building, debugging, deploying .NET, mobile, web, and desktop applications.

✔ What is ILDASM?

A tool used to view IL code, metadata, and assembly structure of .NET EXE/DLL files.

If you want, I can also provide:

- ✔ ILDASM screenshots explanation (text-based)
- ✔ Managed vs Unmanaged short notes
- ✔ Viva questions on CLR and ILDASM
- ✔ .NET Architecture diagram

Just tell me!

Console Applications and Class Libraries.Net Core C# Basics

Here are **simple, clean, exam-oriented notes** on **Console Applications, Class Libraries in .NET Core, and C# Basics —** written in Indian-English style.

✅ 1. Console Applications in .NET Core

A **Console Application** is a C# program that runs in a **command-line window (terminal)**. It is used for simple programs, logic building, backend tasks, and learning C#.

How to Create a Console App (CLI)

bash

Copy code

```
dotnet new console -n MyApp
```

Project Structure

nginx Copy code

```
MyApp
 ├── Program.cs
 ├── MyApp.csproj
```

Program.cs example

csharp Copy code

```csharp
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello .NET Core!");
    }
}
```

How to Run

bash Copy code

```
dotnet run
```

Features

- Fast execution
- Best for algorithms, utility tools, backend scripts
- Pure C# code with no UI
- Supports .NET Core cross-platform execution

✅ 2. Class Libraries in .NET Core

A **Class Library** is a collection of classes, functions, interfaces that can be reused by other applications.

These generate a **DLL file**.

Purpose of Class Libraries

- Reuse code across multiple apps
- Store business logic
- Create APIs, helper methods
- Use in Console, Web, Mobile apps

How to Create a Class Library

bash Copy code

```
dotnet new classlib -n MyLibrary
```

Project Output

Copy code

```
MyLibrary.dll
```

Example Class in Library

csharp Copy code

```csharp
namespace MyLibrary
{
    public class Calculator
    {
        public int Add(int a, int b)
        {
            return a + b;
        }
    }
}
```

Refer Class Library in Console App

1. Go to console app folder
2. Add reference

bash Copy code

```
dotnet add reference ../MyLibrary/MyLibrary.csproj
```

Use the Library

csharp Copy code

```
using MyLibrary;

class Program
{
    static void Main()
    {
        Calculator c = new Calculator();
        Console.WriteLine(c.Add(5, 3));
    }
}
```

✅ 3. C# Basics

A quick guide to basic C# concepts — perfect for exams and interviews.

3.1 C# Program Structure

csharp Copy code

```
using System;

class Program
{
    static void Main()
    {
```

```
        Console.WriteLine("Hello!");
    }
}
```

Explanation

- `using System;` → imports namespaces
- `class Program` → class declaration
- `static void Main()` → program execution starts here

3.2 Variables in C#

Syntax

csharp Copy code

```
dataType variableName = value;
```

Examples

csharp Copy code

```
int age = 20;
string name = "Raj";
double salary = 50000.50;
bool isActive = true;
```

Common Data Types

Type	Description
int	Whole numbers
float	Decimal (low precision)
double	Decimal (high precision)
decimal	Money values
char	Single character

Type	Description
string	Text
bool	true/false

3.3 Operators

Arithmetic

```diff
+ - * / %
```

Comparison

```yaml
== != > < >= <=
```

Logical

```
&& || !
```

Assignment

```
= += -= *= /=
```

3.4 Control Statements

if–else

```csharp
```

```csharp
if (age >= 18)
    Console.WriteLine("Adult");
else
    Console.WriteLine("Minor");
```

switch

csharp Copy code

```csharp
switch(day)
{
    case 1: Console.WriteLine("Monday"); break;
}
```

3.5 Loops

for loop

csharp Copy code

```csharp
for(int i = 1; i <= 5; i++)
    Console.WriteLine(i);
```

while loop

csharp Copy code

```csharp
while(x < 10)
{
    x++;
}
```

foreach loop

csharp Copy code

```csharp
foreach(int n in numbers)
{
```

```
    Console.WriteLine(n);
}
```

3.6 Methods

csharp Copy code

```csharp
int Add(int x, int y)
{
    return x + y;
}
```

3.7 Classes & Objects

csharp Copy code

```csharp
class Person
{
    public string Name;
}

Person p = new Person();
p.Name = "Raj";
```

⭐ Short Exam-Ready Notes

✔ What is a Console App?

A text-based application that runs in the terminal and executes C# code line by line.

✔ What is a Class Library?

A reusable collection of classes and methods compiled into a DLL file.

✔ What is C#?

A modern, object-oriented programming language used to build .NET applications.

If you want, I can also prepare:

- ✔ **MCQs from this topic**
- ✔ **One-page PDF notes**
- ✔ **Assignments / practice questions**
- ✔ **Simple Console + Class Library combined demo project**

Just tell me!

Project References, using
Classes
Data Types in .net and CTS equivalents

Here are clear, structured, **exam-oriented notes** on the topics you asked for:

Project References, `using` Keyword, Classes, and Data Types in .NET (CTS Equivalents)

(C# / .NET Core Study Notes)

---

1. Project References in .NET

What is a Project Reference?

- A **project reference** links one project to another within the same solution.
- If Project A references Project B, then A can access the public classes, functions, and interfaces of B.
- Automatically handles **build order**, ensuring B builds before A.

Why use Project References?

- Encourages **modular programming**.
- Allows **code reuse** across multiple projects.
- Reduces duplication and improves maintainability.

How to add a Project Reference in Visual Studio?

1. Right-click on the project → **Add** → **Project Reference**.
2. Select the project you want to reference.
3. Click **OK**.

Example

Solution contains two projects:

- `LibraryProject`
- `ConsoleApp`

In ConsoleApp:

csharp Copy code

```csharp
using LibraryProject;

class Program
{
    static void Main()
    {
        MyClass obj = new MyClass(); // Accessible because of reference
        obj.Display();
    }
}
```

---

2. `using` Keyword in C#

The `using` keyword has **two major purposes**:

A. Namespace Import

Used to bring namespaces into scope.

csharp Copy code

```csharp
using System;
using ProjectA.Models;
```

Benefits:

- Avoids writing fully qualified names.
- Improves readability.

B. Using Statement (Resource Management)

Used for automatic **dispose** of unmanaged resources.

csharp Copy code

```csharp
using (StreamReader sr = new StreamReader("file.txt"))
{
    Console.WriteLine(sr.ReadLine());
}
```

Why?

- Ensures objects implementing **IDisposable** are cleaned up.
- Prevents memory leaks.

---

3. Classes in C#

Definition

A **class** is a blueprint for creating objects.

It contains:

- Fields
- Properties
- Methods
- Constructors
- Events

Basic Class Example

csharp Copy code

```csharp
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }

    public void Display()
    {
        Console.WriteLine($"Id: {Id}, Name: {Name}");
    }
}
```

Creating an Object

csharp Copy code

```csharp
Student s = new Student();
s.Id = 1;
s.Name = "Raj";
s.Display();
```

4. Data Types in .NET and CTS Equivalents

What is CTS? (Common Type System)

CTS defines **how data types are declared, used, stored** in .NET.

Ensures **language interoperability**.

E.g., a type created in C# can be used in VB.NET or F#.

A. C# Types vs CTS Types

C# Keyword	.NET CTS Type	Size	Description
`bool`	`System.Boolean`	1 byte	True/False
`byte`	`System.Byte`	1 byte	Unsigned byte
`sbyte`	`System.SByte`	1 byte	Signed byte
`short`	`System.Int16`	2 bytes	Signed integer
`ushort`	`System.UInt16`	2 bytes	Unsigned integer
`int`	`System.Int32`	4 bytes	Signed integer
`uint`	`System.UInt32`	4 bytes	Unsigned int
`long`	`System.Int64`	8 bytes	Signed long
`ulong`	`System.UInt64`	8 bytes	Unsigned long
`float`	`System.Single`	4 bytes	Floating point
`double`	`System.Double`	8 bytes	Double precision
`decimal`	`System.Decimal`	16 bytes	High precision
`char`	`System.Char`	2 bytes	Unicode character
`string`	`System.String`	Variable	Sequence of chars
`object`	`System.Object`	N/A	Base of all types

Value Types vs Reference Types

Value Types

- Stored in **stack**
- Contain actual data
- Examples: `int` , `float` , `bool` , `struct` , `enum`

Reference Types

- Stored in **heap**
- Store reference (address)
- Examples: `class` , `string` , `array` , `interface`

---

5. Common Language Runtime (CLR) and Data Types

CLR ensures:

- Type safety
- Memory management
- Garbage collection
- Inter-language compatibility
  All languages compile into **MSIL (Intermediate Language)**.

If you want, I can continue with:

- ✔ Constructors
- ✔ Access Modifiers
- ✔ OOP Concepts
- ✔ Structs vs Classes
- ✔ Interfaces
- ✔ Namespaces
- ✔ Inheritance & Polymorphism

Just tell me "**Next topic**" or mention the topic name.

Methods
 MethodOverloading
 OptionalParameters
 NamedParametersandPositional Parameters

- Usingparams
- Localfunctions

Here are **clean, simple, exam-oriented notes** on **Methods, Method Overloading, Optional Parameters, Named & Positional Parameters,** `params` **, and Local Functions —** perfect for .NET / C# theory and viva.

---

1. Methods

Definition

A method is a **block of code** that performs a task when called.

Syntax:

csharp Copy code

```csharp
returnType MethodName(parameters)
{
    // method body
}
```

Example:

csharp Copy code

```csharp
void Greet()
{
    Console.WriteLine("Hello!");
}
```

---

2. Method Overloading

Definition

Method Overloading means **multiple methods with the same name** but:

- Different number of parameters, OR
- Different types of parameters, OR
- Different order of parameters

Why?

- Improves readability
- Supports compile-time polymorphism

Example:

csharp Copy code

```csharp
public void Add(int a, int b)
{
    Console.WriteLine(a + b);
}

public void Add(double a, double b)
{
    Console.WriteLine(a + b);
}

public void Add(int a, int b, int c)
{
    Console.WriteLine(a + b + c);
}
```

---

3. Optional Parameters

Definition

A parameter with a **default value** → caller may skip it.

Syntax:

csharp Copy code

```csharp
void Display(string name = "Guest")
{
```

```
    Console.WriteLine("Hello " + name);
}
```

Usage:

csharp Copy code

```
Display();            // Output: Hello Guest
Display("Raj");        // Output: Hello Raj
```

---

4. Named Parameters & Positional Parameters

A. Positional Parameters

Arguments passed **in order**.

csharp Copy code

```
void Print(int a, int b)
{
    Console.WriteLine(a + ", " + b);
}

Print(10, 20); // Positional
```

B. Named Parameters

Arguments passed **by specifying names**, independent of order.

csharp Copy code

```
Print(b: 20, a: 10);
```

Benefits:

- Improves readability
- Useful when many parameters exist, especially optional ones

---

5. Using `params`

Definition

Allows passing **variable number of arguments** to a method.

Syntax:

csharp Copy code

```csharp
void PrintNumbers(params int[] numbers)
{
    foreach (int n in numbers)
        Console.WriteLine(n);
}
```

Usage:

csharp Copy code

```csharp
PrintNumbers(1, 2, 3, 4, 5);

int[] arr = { 10, 20, 30 };
PrintNumbers(arr);
```

Rules:

- Only **one** `params` parameter per method.
- Must be the **last** parameter.

---

6. Local Functions

Definition

A **function inside another method**.

Used for helper logic that is only required within that method.

Benefits:

- Encapsulation
- Cleaner code

- Avoids unnecessary separate private methods

Example:

csharp

Copy code

```csharp
void ProcessData(int x)
{
    Console.WriteLine("Processing...");

    // Local function
    int Square(int n)
    {
        return n * n;
    }

    Console.WriteLine("Square: " + Square(x));
}

ProcessData(5);
```

Quick Revision Table

Concept	Meaning	Key Point
Method	Block of code performing task	Can return value or not
Method Overloading	Same method name, different signature	Compile-time polymorphism
Optional Parameters	Default values provided	Can skip arguments
Named Parameters	Pass arguments with names	Order doesn't matter
Positional Parameters	Arguments passed by order	Default method calling
`params`	Variable number of arguments	Must be last parameter

Concept	Meaning	Key Point
Local Functions	Function inside method	Scope limited to method

---

If you want, I can continue with:

- ✔ Constructors
- ✔ Access Modifiers
- ✔ Static vs Instance Methods
- ✔ Ref & Out Parameters

Just tell me the next topic!

Properties
 get,set
 Readonlyproperties
 Usingproperty accessors to create Readonly property

Below are **clear, easy, exam-oriented notes** on **Properties in C#**, including **get/set**, **read-only properties**, and **how to use accessors to create them**.

---

1. Properties in C#

Definition

A **property** is a class member that provides a **flexible mechanism to read, write, or compute** the value of a private field.

- Used to **control access** to class fields
- Provides **encapsulation**
- Uses **get** and **set** accessors

Syntax of a Property

csharp Copy code

```csharp
class Person
{
    private string name;   // private field

    public string Name     // property
    {
        get { return name; }
        set { name = value; }
    }
}
```

---

2. get and set Accessors

✔ get

- Returns the value of the property
- Acts like a method without parameters
- Should never modify data

✔ set

- Assigns value to the property
- Uses an implicit variable **value**
- Can include validation logic

Example:

csharp Copy code

```csharp
public int Age
{
    get { return age; }
    set
    {
        if (value >= 0)
            age = value;
```

```
    }
}
```

---

3. Auto-Implemented Properties

Shortcut when no custom logic is needed.

csharp Copy code

```csharp
public string Email { get; set; }
```

Compiler creates a **hidden backing field**.

---

4. Read-Only Properties

These are properties that can **only be read**, not modified.

Type 1: Read-only by omitting `set`

csharp Copy code

```csharp
public string Id
{
    get { return id; }  // only getter
}
```

Setter is not provided → cannot assign from outside.

Type 2: Auto-implemented Read-only Properties (C# 6+)

csharp Copy code

```csharp
public string OrderId { get; }
```

The value can only be assigned:

- In the **constructor**

- Or during declaration

csharp Copy code

```csharp
public class Order
{
    public string OrderId { get; }

    public Order()
    {
        OrderId = Guid.NewGuid().ToString();
    }
}
```

---

5. Creating Read-Only Properties Using Access Modifiers

You can make the property writable **inside the class**, but read-only **outside**.

Example:

csharp Copy code

```csharp
public int Marks { get; private set; }
```

- `private set` → only internal class code can assign
- External code can only `get`

Usage:

csharp Copy code

```csharp
class Student
{
    public int Marks { get; private set; }

    public void AssignMarks(int m)
    {
        Marks = m;    // allowed
    }
}
```

```
Student s = new Student();
// s.Marks = 90;    ❌ Not allowed
```

---

6. Write-Only Properties (rare)

C# allows a property with only a `set`.

csharp Copy code

```csharp
public string Password
{
    set { _password = Encrypt(value); }
}
```

---

7. Key Differences

Property Type	get	set	Assigned From
Read-Write	✔	✔	Outside + Inside
Read-Only	✔	✖	Constructor/Inside
Write-Only	✖	✔	Outside
Private Set	✔	✔ (private)	Inside only

---

8. Example: Full Property with Validation

csharp Copy code

```csharp
private int _age;

public int Age
{
    get { return _age; }
```

```
        set
        {
            if (value < 0)
                throw new Exception("Invalid Age");
            _age = value;
        }
    }
```

---

9. Best Practices

- ✔ Use **auto-properties** for simple cases
- ✔ Use **validation** inside setters
- ✔ Use **read-only (get only)** properties for IDs, timestamps
- ✔ Use **private set** for controlled updates

If you want next topic, I can explain:

- ✔ Constructors (default, parameterized, static)
- ✔ Object and Class Deep Concepts
- ✔ Fields vs Properties differences

Just tell me!

Constructors
Object Initializer
Destructors
Discussion on IDispose. To be implemented after interfaces

Here are **clean, exam-oriented notes** on **Constructors, Object Initializers, Destructors, and IDisposable** in C#—written in simple Indian-English.

⭐ 1. Constructors

Definition

A **constructor** is a special method that is automatically called when an object is created.

Characteristics:

- Has **same name** as class
- **No return type**
- Used to initialize fields
- Can be **overloaded**

Types of Constructors

✔ 1. Default Constructor

Provided by compiler if no constructor is written.

csharp Copy code

```csharp
class Student
{
    public Student()
    {
        Console.WriteLine("Default Constructor Called");
    }
}
```

✔ 2. Parameterized Constructor

Used to pass values while creating object.

csharp Copy code

```csharp
class Student
{
    public string Name;
```

```
    public Student(string name)
    {
        Name = name;
    }
}
```

Usage:

csharp Copy code

```
Student s = new Student("Raj");
```

✔ 3. Copy Constructor (C# doesn't provide built-in, but we can create manually)

csharp Copy code

```
public Student(Student s)
{
    this.Name = s.Name;
}
```

✔ 4. Static Constructor

- Runs **once per class**
- Used to initialize static fields
- No parameters, no access modifiers

csharp Copy code

```
static Student()
{
    Console.WriteLine("Static Constructor");
}
```

✔️ 5. Private Constructor

Used in **Singleton pattern** or to prevent object creation.

csharp Copy code

```
private Student() { }
```

⭐ 2. Object Initializer

Definition

Object Initializers allow you to initialize object **without calling constructor explicitly**, or **along with constructor**, using property syntax.

Example:

csharp Copy code

```
Student s = new Student
{
    Name = "Raj",
    Age = 22
};
```

With constructor:

csharp Copy code

```
Student s = new Student("Mumbai")
{
    Name = "Raj",
    Age = 22
};
```

Benefits:

- Clean syntax
- Avoids writing multiple overloaded constructors
- Good for model classes in ASP.NET / EF

⭐ 3. Destructors

Definition

A destructor is used to perform cleanup **just before object is destroyed**.

Syntax:

csharp Copy code

```
~Student()
{
    Console.WriteLine("Destructor Called");
}
```

Key Points:

- Only **one destructor** per class
- Cannot take parameters
- Cannot be overloaded
- Called by **Garbage Collector (GC)** automatically
- Used rarely in modern C# because GC handles memory

Important:

Destructors **do NOT guarantee immediate cleanup**.

⭐ 4. IDisposable Interface

Used to release **unmanaged resources** manually such as:

- File handles
- Network connections
- Database connections
- OS-level resources

✔ IDisposable Definition

IDisposable contains a single method:

csharp Copy code

```csharp
public interface IDisposable
{
    void Dispose();
}
```

✔ Why IDisposable?

Because GC only cleans **managed memory**, not unmanaged resources.

`Dispose()` allows **deterministic cleanup**.

✔ Implementing IDisposable

csharp Copy code

```csharp
class FileHandler : IDisposable
{
    private FileStream fs;

    public FileHandler(string path)
    {
        fs = new FileStream(path, FileMode.Open);
    }

    public void Dispose()
    {
        fs.Close();
        Console.WriteLine("Resources Released");
    }
}
```

⭐ Using IDisposable with using Statement

The `using` keyword automatically calls `Dispose()`.

csharp Copy code

```csharp
using (FileHandler fh = new FileHandler("data.txt"))
{
    // work with fh
}
// Dispose() is called automatically here
```

⭐ Destructor + IDisposable Together (Advanced Pattern)

Common for classes holding unmanaged resources.

csharp Copy code

```csharp
class Sample : IDisposable
{
    ~Sample()
    {
        Dispose();
    }

    public void Dispose()
    {
        GC.SuppressFinalize(this);  // Stop destructor from running again
        Console.WriteLine("Cleaned up");
    }
}
```

⭐ Summary Table

Concept	Purpose	Managed by
Constructor	Initializes object	Developer
Object Initializer	Easy property assignment	Developer

Concept	Purpose	Managed by
Destructor	Cleanup before GC destroys object	GC
IDisposable	Manual cleanup of unmanaged resources	Developer

If you want, I can continue with:

- ✔ Interfaces
- ✔ Inheritance
- ✔ Polymorphism
- ✔ Abstract classes

Just tell me the next topic!

Constructors
Object Initializer
Destructors
Discussion on IDispose. To be implemented after interfaces

Below is a **clean, structured, exam-oriented explanation** of **Constructors, Object Initializers, Destructors, and IDisposable**—exactly as required.
(*You said "IDispose to be implemented after Interfaces," so I have included IDisposable concept but left full implementation for when we cover Interfaces.*)

⭐ 1. Constructors

Definition

A **constructor** is a special method that is executed **when an object is created**.

Purpose: **Initialize fields** or perform startup logic.

Key Characteristics

- Same name as class
- No return type (not even void)
- Called automatically
- Can be overloaded

Types of Constructors in C#

✔ (a) Default Constructor

Provided by the compiler if no constructor is written.

csharp Copy code

```csharp
public class Student
{
    public Student()
    {
        Console.WriteLine("Default Constructor Called");
    }
}
```

✔ (b) Parameterized Constructor

csharp Copy code

```csharp
public class Student
{
    public string Name;
    public Student(string name)
    {
        Name = name;
    }
}
```

Used for **passing values at the time of object creation**.

✔ (c) Static Constructor

Used to initialize **static data**.

- Runs **once per class**
- No parameters
- No access modifiers

csharp Copy code

```csharp
static Student()
{
    Console.WriteLine("Static Constructor");
}
```

✔ (d) Private Constructor

Used to **restrict object creation** and in patterns like Singleton.

csharp Copy code

```csharp
private Student() { }
```

✔ (e) Copy Constructor (user-defined)

csharp Copy code

```csharp
public Student(Student s)
{
    this.Name = s.Name;
}
```

⭐ 2. Object Initializer

Definition

Object Initializers allow property assignment **without explicitly calling multiple constructors**.

Example:

csharp Copy code

```csharp
var student = new Student
{
    Name = "Raj",
    Age = 22
};
```

With constructor:

csharp Copy code

```csharp
var product = new Product(100)
{
    Name = "Laptop",
    Price = 75000
};
```

Advantages

- Cleaner syntax
- Avoids too many constructor overloads
- Easy to read and maintain

⭐ 3. Destructors

Definition

A destructor is used for **cleanup actions** before an object is destroyed by the Garbage Collector.

Syntax:

csharp

```
~Student()
{
    Console.WriteLine("Destructor Called");
}
```

Important Points:

- Only **one destructor** per class
- No parameters
- Cannot be overloaded
- Called automatically by **Garbage Collector**
- No guarantee *when* it will run

➡️ Destructors are **rarely used** in modern .NET (GC handles most cleanup).

⭐ 4. IDisposable Interface (Concept Only for Now)

(*Full implementation will be shown after Interfaces chapter, as per your instruction.*)

Why IDisposable?

Because Garbage Collector cleans up **managed memory**,

but **unmanaged resources** must be released manually, such as:

- File handles
- Database connections
- Network sockets
- OS-level resources

IDisposable – The Cleanup Contract

IDisposable only has **one method**:

csharp Copy code

```
void Dispose();
```

Any class that holds unmanaged resources **should implement IDisposable**.
This gives programmers control over **immediate cleanup**.

Using Disposable Objects

Even without showing implementation, you must understand:

csharp Copy code

```csharp
using (var file = new FileStream("data.txt", FileMode.Open))
{
    // use file
}       // Dispose() is called automatically here
```

`using` block ensures cleanup even if an exception occurs.

⭐ Difference Between Destructor & IDisposable

Feature	Destructor	IDisposable
Who calls it?	Garbage Collector	Developer / using block
When?	Unknown, non-deterministic	Immediate, predictable
Purpose	Cleanup before GC	Cleanup unmanaged resources
Syntax	~ClassName()	void Dispose()

⭐ Summary

Topic	Purpose
Constructor	Initialize object
Object Initializer	Assign properties cleanly
Destructor	Final cleanup before GC

Topic	Purpose
IDisposable	Deterministic cleanup of unmanaged resources

If you want, I can proceed with the next chapter:

- ✔ **Interfaces**
- ✔ **Inheritance**
- ✔ **Abstract classes**
- ✔ **Polymorphism**

Just tell me the next heading!

Static Members of a Class
□ Fields
□ Methods
□ Properties
□ Constructors

Here are **simple, clean, exam-oriented notes** on **Static Members of a Class** in C# — **Fields, Methods, Properties, Constructors** — perfect for viva, theory, and practical understanding.

⭐ Static Members of a Class (C#)

Definition

`static` members belong to the **class itself**, not to any object.

- Shared among **all objects**
- Accessed using **ClassName.MemberName**

- Loaded into memory **once** when the program starts

⭐ 1. Static Fields

Definition

A static field is shared by all objects of a class.

csharp Copy code

```csharp
class Student
{
    public static int Count = 0;
}
```

Characteristics:

- Only **one copy** exists for the entire class
- Not dependent on object creation
- Access using class name:

csharp Copy code

```csharp
Student.Count = 10;
```

⭐ 2. Static Methods

Definition

Methods that belong to the class instead of an object.

csharp Copy code

```csharp
class Calculator
{
    public static int Add(int a, int b)
    {
        return a + b;
```

```
    }
}
```

Calling:

csharp Copy code

```
int result = Calculator.Add(10, 20);
```

Characteristics:

- ✔ Cannot use **non-static** fields inside static methods
- ✔ Can only access **static fields/methods**
- ✔ Commonly used for:
  - Utility functions
  - Helper classes
  - Math operations

⭐ 3. Static Properties

Definition

Properties declared with the `static` keyword.

csharp Copy code

```
class Config
{
    public static string AppName { get; set; } = "MyApp";
}
```

Usage:

csharp Copy code

```
Console.WriteLine(Config.AppName);
```

Purpose:

- Shared configuration
- Constants
- Settings

⭐ 4. Static Constructors

Definition

A constructor that initializes **static members** of a class.

Syntax:

csharp Copy code

```csharp
class Demo
{
    static Demo()
    {
        Console.WriteLine("Static Constructor Called");
    }
}
```

Key Characteristics:

- Automatically called **once** before first use of the class
- No parameters allowed
- No access modifiers
- Runs only **once per application**

Triggered when:

- First object is created OR
- First time accessing a static member

⭐ Summary Table

Static Member	What it Belongs To	Purpose	Example
Static Field	Class	Shared data	`Student.Count`

Static Member	What it Belongs To	Purpose	Example
Static Method	Class	Utility logic	`Calculator.Add()`
Static Property	Class	Shared read/write values	`Config.AppName`
Static Constructor	Class	Initialize static members	Runs once automatically

---

⭐ Small Example Showing All Static Members

csharp Copy code

```csharp
class Bank
{
    public static double InterestRate;      // static field

    static Bank()                           // static constructor
    {
        InterestRate = 7.5;
    }

    public static void ShowRate()           // static method
    {
        Console.WriteLine("Rate is " + InterestRate);
    }

    public static double Rate               // static property
    {
        get { return InterestRate; }
        set { InterestRate = value; }
    }
}
```

Usage:

csharp Copy code

```
Bank.ShowRate();
Bank.Rate = 8.0;
```

If you want, I can explain next topics:

- ✔ Static vs Instance Members
- ✔ Inheritance
- ✔ Interfaces
- ✔ Polymorphism

Just tell me the next heading!

Static Classes
Static local functions

Below are **clean, clear, classroom-style notes** on **Static Classes** and **Static Local Functions** in C#. Perfect for theory, viva, and practical use.

⭐ 1. Static Classes

Definition

A **static class** is a class that:

- Cannot be instantiated (no objects can be created)
- Contains only **static members**
- Is loaded once into memory

Example:

csharp

Copy code

```csharp
static class MathHelper
{
    public static int Square(int x) => x * x;
}
```

Usage:

csharp Copy code

```csharp
int result = MathHelper.Square(5);
```

⭐ Characteristics of Static Classes

Feature	Description
Instantiation	❌ Cannot create objects
Constructors	✔️ Only **static constructor** allowed
Members	✔️ Must be static only
Inheritance	❌ Cannot inherit or be inherited from
Purpose	Helper/utility/common methods

✔️ Advantages

- Better performance (loaded once)
- Ensures no accidental object creation
- Good for **utility**, **helper**, **constant**, **configuration** classes

---

✔️ Example: Static Utility Class

csharp Copy code

```csharp
static class StringUtils
{
    public static bool IsEmpty(string s)
    {
        return string.IsNullOrWhiteSpace(s);
    }
}
```

Usage:

csharp Copy code

```csharp
bool check = StringUtils.IsEmpty("hello");
```

⭐ 2. Static Local Functions (C# 8+)

Definition

A **local function** is a function declared *inside another method*.

A **static local function**:

- Cannot access **instance members**
- Cannot access **local variables** of the outer method
- Can access **static members**
- Safer and faster because it avoids capturing the outer scope

---

✔️ Example of Static Local Function

csharp Copy code

```csharp
void ProcessNumbers()
{
    int result = Multiply(10, 20);
    Console.WriteLine(result);

    static int Multiply(int x, int y)  // static local function
    {
```

```
        return x * y;
    }
}
```

⭐ Why Use Static Local Functions?

✔ 1. Performance

Prevents "closure capturing" — avoids hidden memory allocation.

✔ 2. Safety

Compiler guarantees the function:

- does NOT use outer local variables
- does NOT use `this` (instance data)
- is purely independent

✔ 3. Readability

Useful when:

- Function is only required inside one method
- Should not be exposed outside

⭐ Difference Between Local Function & Static Local Function

Feature	Local Function	Static Local Function
Access outer variables	✔ Allowed	❌ Not allowed
Can use `this`	✔ Yes	❌ No
Performance	Slower (due to capturing)	Faster
Safety	Less strict	Very strict
Typical use	Accessing method data	Independent logic

✔️ Example: When Static Local Function is NOT Allowed

csharp Copy code

```csharp
void Show()
{
    int a = 10;

    static void Test() // ❌ ERROR
    {
        Console.WriteLine(a); // Cannot access outer variable
    }
}
```

`static` local function cannot access `a`.

⭐ Combined Example

csharp Copy code

```csharp
static class Calculator
{
    public static void Work()
    {
        Console.WriteLine(Add(10, 20));

        static int Add(int a, int b)   // static local function
        {
            return a + b;
        }
    }
}
```

⭐ Summary

Topic	Meaning
Static Class	Class with only static members, no object creation
Static Local Function	Local method inside another method that cannot capture surrounding variables

---

If you want next topics, I can continue with:

- ✔ Interfaces
- ✔ Inheritance
- ✔ Partial classes
- ✔ Structs & Enums
- ✔ Delegates & Events

Just tell me the next heading!

Inheritance

- □ AccessSpecifiers
- □ Constructors in a hierarchy
- □ Overloading in derived class
- □ Hiding, using new
- □ override
- □ sealedmethods
- □ AbstractClasses
- □ AbstractMethods
- □ SealedClasses

Here are **clean, structured, exam-oriented notes** on **Inheritance in C#**, covering all the topics you listed:

Inheritance in C#

Inheritance allows a class (derived/child class) to acquire the **fields, methods, and properties** of another class (base/parent class).
It promotes **code reusability** and **polymorphism**.

---

1. Access Specifiers in Inheritance

These determine how class members are accessible in derived classes.

Access Specifier	Accessible in Same Class	Derived Class	Other Assembly
public	✔	✔	✔
private	✔	✘	✘
protected	✔	✔	✘
internal	✔	✔ (same assembly)	✘
protected internal	✔	✔ (same assembly or via inheritance)	✘
private protected	✔	✔ (same assembly + inheritance only)	✘

---

2. Constructors in a Hierarchy

Rules:

- ✔ Base class constructor executes **before** derived class constructor
- ✔ Derived class can call a specific base constructor using `: base(...)`

Example

csharp

Copy code

```csharp
class A
{
    public A() { Console.WriteLine("A default"); }
}

class B : A
{
    public B() : base()
    {
        Console.WriteLine("B default");
    }
}
```

---

3. Overloading in Derived Class

Derived class can define multiple constructors or methods with the same name but different parameters.

Overloading does **not** affect base methods unless overridden.

---

4. Hiding Members (using new)

If a derived class defines a member with the **same name** as the base class, it hides the base member.

Use **new** keyword to explicitly hide:

csharp Copy code

```csharp
class A
{
    public void Show() => Console.WriteLine("A Show");
}

class B : A
{
```

```
    public new void Show() => Console.WriteLine("B Show");
}
```

❗ Hiding ≠ Overriding

👉 It depends on **reference type** (A or B).

---

5. override (Method Overriding)

Used for **runtime polymorphism**.

Requirements:

- Base method must be marked **virtual**
- Derived method must use **override**

csharp Copy code

```
class A
{
    public virtual void Show() { Console.WriteLine("A Show"); }
}

class B : A
{
    public override void Show() { Console.WriteLine("B Show"); }
}
```

✔️ Decided at runtime

✔️ Depends on **object type**, not reference type.

---

6. sealed Methods

Prevents further overriding.

csharp Copy code

```
class A
{
```

```
    public virtual void Show() { }
}

class B : A
{
    public sealed override void Show() { }
}
```

Now no class can override `Show()` from B.

---

7. Abstract Classes

An abstract class:

- **Cannot be instantiated**
- Can have **abstract and non-abstract methods**
- Can contain fields, properties, constructors

csharp Copy code

```
abstract class Shape
{
    public abstract void Draw();
}
```

---

8. Abstract Methods

- Declared with **no body**
- Must be **overridden** in derived classes

csharp Copy code

```
abstract class Shape
{
    public abstract void Draw();
}

class Circle : Shape
```

```
{
    public override void Draw() => Console.WriteLine("Drawing Circle");
}
```

---

9. Sealed Classes

A **sealed class** cannot be inherited.

csharp Copy code

```
sealed class A
{
    public void Show() { }
}
```

Mostly used for:

- Security
- Performance optimization

Complete Example Showing All Concepts

csharp Copy code

```
abstract class Animal
{
    public Animal() { Console.WriteLine("Animal created"); }

    public virtual void Speak() => Console.WriteLine("Animal sound");

    public abstract void Eat();
}

class Dog : Animal
{
    public Dog() : base() { Console.WriteLine("Dog created"); }

    public override void Speak() => Console.WriteLine("Bark");
```

```
    public override void Eat() => Console.WriteLine("Dog eats bones");

    public new void Run() => Console.WriteLine("Dog runs");
}

sealed class PetDog : Dog
{
    public sealed override void Speak() => Console.WriteLine("Pet dog bark softly
}
```

Would you like next topics?

I can also prepare notes for:

✅ Interfaces
✅ Polymorphism
✅ Delegates & Events
✅ Exception Handling

Or anything you need for C#, .NET or SQL.

Just tell me!

Interfaces
□ Implementing aninterface
□ Explicitly implementing an interface
□ Inheritance in interfaces
□ Defaultinterface methods

Here are **clean, structured, exam-oriented notes** on **Interfaces in C#**, covering all the topics you listed.

Interfaces in C#

An **interface** is a contract that contains **only definitions** (method signatures, properties, events, indexers).
A class or struct that implements an interface must provide the **full implementation**.

- ✔ Supports **multiple inheritance**
- ✔ Cannot contain fields
- ✔ Members are **public** by default (until C# 8: now default methods allowed)

---

1. Implementing an Interface

A class must implement **all** members of the interface.

Example

csharp Copy code

```csharp
interface IShape
{
    void Draw();
}

class Circle : IShape
{
    public void Draw()
    {
        Console.WriteLine("Drawing Circle");
    }
}
```

Key points:

- Methods in interface = **no body** (unless default method)
- Implementing class must mark methods **public**

---

2. Explicit Interface Implementation

Used when:

- Multiple interfaces have **same method name**
- You want to **hide interface method** from normal object calls

Example

csharp Copy code

```csharp
interface I1 { void Show(); }
interface I2 { void Show(); }

class Demo : I1, I2
{
    void I1.Show()
    {
        Console.WriteLine("I1 Show");
    }

    void I2.Show()
    {
        Console.WriteLine("I2 Show");
    }
}
```

Usage:

csharp Copy code

```csharp
Demo d = new Demo();
// d.Show(); ❌ NOT allowed

((I1)d).Show();  // ✓ I1 Show
((I2)d).Show();  // ✓ I2 Show
```

✔ Useful when two interfaces have conflicting method names
✔ Explicit methods are **not accessible** using class reference

---

3. Inheritance in Interfaces

Interfaces can inherit from other interfaces.

Example

csharp Copy code

```csharp
interface IA
{
    void A();
}

interface IB : IA
{
    void B();
}

class Test : IB
{
    public void A() { }
    public void B() { }
}
```

Key Points:

- A class implementing IB must implement **both** A() and B()
- Supports **multiple inheritance**:

csharp Copy code

```csharp
interface IC : IA, IB { }
```

---

4. Default Interface Methods (C# 8+)

Interfaces can have **methods with body**, similar to Java's default methods.

Purpose:

- Add new functionality **without breaking existing classes**

Example

csharp Copy code

```csharp
interface ILog
{
    void Log(string msg);

    // Default method
    void LogWarning(string msg)
    {
        Console.WriteLine("WARNING: " + msg);
    }
}

class FileLogger : ILog
{
    public void Log(string msg)
    {
        Console.WriteLine("File Log: " + msg);
    }
}
```

Usage:

csharp Copy code

```csharp
ILog logger = new FileLogger();
logger.Log("Test");
logger.LogWarning("Low disk space");  // default method runs
```

- ✔ Class can override default method
- ✔ Cannot define instance fields
- ✔ Not allowed in earlier versions of .NET

Complete Example Covering All Concepts

csharp Copy code

```csharp
interface IAnimal
{
    void Eat();
```

```
    // Default method
    void Sleep()
    {
        Console.WriteLine("Animal sleeping");
    }
}

interface IPet : IAnimal
{
    void Play();
}

class Dog : IPet
{
    public void Eat() => Console.WriteLine("Dog eats");

    public void Play() => Console.WriteLine("Dog plays");

    // Optional override
    public void Sleep() => Console.WriteLine("Dog sleeps quietly");
}
```

Key Differences — Interface vs Abstract Class

Interface	Abstract Class
No fields	Can have fields
Supports multiple inheritance	Cannot use multiple inheritance
Default implementations allowed (C# 8+)	Methods may have implementation
Members are public by default	Members can have any access modifier
Cannot have constructors	Can have constructors

If you want, I can prepare the next topics:

✅ Delegates & Events
✅ Generics
✅ Exception Handling
✅ LINQ

Just tell me!

Operator overloading

Here are **simple, clean, interview-oriented notes** on **Operator Overloading in C#**, with examples and rules.

⭐ Operator Overloading in C#

Operator Overloading allows you to **redefine how operators work** for your custom classes and structs.

Example:
`+ , -, *, /, ==, >, <, []` etc. can be overloaded.

✔ Improves readability
✔ Makes user-defined types behave like built-in types

⭐ Syntax

Operator overloading is done using the keyword **operator** inside a class/struct.

csharp Copy code

```csharp
public static ReturnType operator op(Type a, Type b)
{
```

```
    // implementation
}
```

⭐ Example: Overloading the + Operator

csharp Copy code

```csharp
class Complex
{
    public int Real;
    public int Imag;

    public Complex(int r, int i)
    {
        Real = r;
        Imag = i;
    }

    public static Complex operator +(Complex c1, Complex c2)
    {
        return new Complex(c1.Real + c2.Real, c1.Imag + c2.Imag);
    }
}
```

Usage:

csharp Copy code

```csharp
var c1 = new Complex(2, 3);
var c2 = new Complex(4, 5);
var c3 = c1 + c2;   // Calls operator overloading
```

⭐ Operators That CAN Be Overloaded

Category	Operators
Arithmetic	`+ - * / %`

Category	Operators
Unary	`+ - ! ~ ++ --`
Relational	`== != > < >= <=`
Logical	`&`
Assignment	None
Indexer	`[]` (via indexer, not operator overloading)
Other	`true , false`

✔️ Some operators **must** be overloaded in pairs:

- `==` with `!=`
- `>` with `<`
- `>=` with `<=`

⭐ Unary Operator Overloading Example (++)

csharp Copy code

```csharp
class Counter
{
    public int Value;

    public Counter(int v) { Value = v; }

    public static Counter operator ++(Counter c)
    {
        return new Counter(c.Value + 1);
    }
}
```

Usage:

csharp Copy code

```
Counter c = new Counter(10);
Counter d = ++c;   // returns new Counter(11)
```

⭐ Overloading == and !=

When overloading relational operators, you should also override **Equals()** and **GetHashCode()**.

csharp Copy code

```
class Point
{
    public int X, Y;

    public static bool operator ==(Point p1, Point p2)
        => p1.X == p2.X && p1.Y == p2.Y;

    public static bool operator !=(Point p1, Point p2)
        => !(p1 == p2);
}
```

⭐ Rules of Operator Overloading

- ✔ Operator must be **public and static**
- ✔ At least one operand must be a **class or struct**
- ✔ Cannot overload:
  - `new`
  - `.` (dot)
  - `?:` (ternary)
  - `=` (assignment)
  - `&&`, `||` (shortcut operators)
- ✔ Overloading does **not** change precedence or number of operands

---

⭐ When to Use Operator Overloading

Use operator overloading when your type behaves like a mathematical or logical value:

- ✔ Complex numbers
- ✔ Vectors, Points
- ✔ Fractions
- ✔ Matrices
- ✔ Money/Currency objects

Avoid it when it reduces readability.

⭐ Summary

- ✔ Operator overloading lets you redefine operators for custom types
- ✔ Must be **public static**
- ✔ Supports arithmetic, relational, unary, logical operators
- ✔ Use when it improves clarity and mathematical interpretation

---

If you want, I can also give:

- 👉 A big **real-life example**
- 👉 Short **exam questions**
- 👉 Practical **coding exercises**

Just tell me!

Reference and Value Types
Value Types
□ struct
□ enum
out and ref
nullable types

nullable reference types
?? and ??=
Working with Arrays (single, multidim, jagged), Array Class members
Indices and ranges
Indexers

Here are **clean, structured, exam-oriented notes** covering **Reference & Value Types**, **struct**, **enum**, **out/ref**, **nullable types**, **?? / ??=**, **arrays**, **indices & ranges**, and **indexers** — all in one place.

⭐ 1. Reference Types vs Value Types

Value Types

Stored in **stack**

Contain the actual data

Copied **by value**

Cannot be null (except nullable types)

Examples:

```
int, float, bool, char, struct, enum
```

Reference Types

Stored in **heap** (variable contains reference)

Copied **by reference**

Two variables can point to the **same object**

Examples:

```
class, string, array, delegate, interface, dynamic
```

⭐ 2. Value Types in Detail

struct

A lightweight value type (similar to class but value-based).

Key Points:

- ✔ Stored in stack
- ✔ Cannot have parameterless constructor (until C# 10)
- ✔ Cannot be inherited (sealed by default)
- ✔ Can implement interfaces
- ✔ Good for small data objects (Point, Coordinate, Color)

Example:

csharp Copy code

```csharp
struct Point
{
    public int X;
    public int Y;

    public Point(int x, int y)
    {
        X = x; Y = y;
    }
}
```

⭐ 3. enum

Represents a set of named constants.

csharp Copy code

```csharp
enum Days { Sun, Mon, Tue, Wed, Thu, Fri, Sat }
```

Underlying type = int (default)

Usage:

csharp Copy code

```
Days d = Days.Mon;
int x = (int)Days.Wed;   // 3
```

⭐ 4. out and ref

ref

- Variable **must be initialized** before passing
- Allows modification inside method

csharp Copy code

```
void Change(ref int x)
{
    x = 100;
}
```

Call:

csharp Copy code

```
int a = 10;
Change(ref a);
```

---

out

- Variable **need NOT be initialized**
- Method **must assign value**

csharp Copy code

```
void GetValues(out int x, out int y)
{
    x = 10; y = 20;
}
```

Usage:

csharp Copy code

```csharp
int a, b;
GetValues(out a, out b);
```

⭐ 5. Nullable Types (Value Types)

Allows value types to hold `null`.

csharp Copy code

```csharp
int? x = null;
bool? flag = null;
```

Check value:

csharp Copy code

```csharp
if(x.HasValue) ...
int y = x.GetValueOrDefault(0);
```

Short form:

`Nullable<int> = int?`

⭐ 6. Nullable Reference Types (C# 8+)

Enable compiler warnings for potential `null` issues.

csharp Copy code

```csharp
string? name = null;      // nullable reference
string address;           // non-nullable reference
```

Prevents **null reference exceptions**.

Enabled using:

csharp Copy code

```
#nullable enable
```

⭐ 7. Null-Coalescing Operators

?? Operator

Returns left value if **not null**, else right.

csharp Copy code

```
string? name = null;
string result = name ?? "Guest";   // "Guest"
```

??= Operator

Assigns a default only if variable is null.

csharp Copy code

```
string? msg = null;
msg ??= "Hello";    // msg = "Hello"
```

⭐ 8. Working with Arrays

1. Single-Dimensional Array

csharp Copy code

```
int[] arr = { 10, 20, 30 };
```

2. Multi-Dimensional Array (Rectangular)

csharp Copy code

```
int[,] matrix = new int[2,3];
```

```
matrix[0,1] = 5;
```

3. Jagged Array (Array of Arrays)

csharp Copy code

```
int[][] jag = new int[3][];
jag[0] = new int[] {1,2};
jag[1] = new int[] {3,4,5};
jag[2] = new int[] {6};
```

⭐ 9. Array Class Members

Useful static methods:

Method	Description
`Sort()`	Sort array
`Reverse()`	Reverse array
`Copy()`	Copy elements
`IndexOf()`	First matching index
`Clear()`	Set elements to default
`Resize()`	Change array size

Example:

csharp Copy code

```
Array.Sort(arr);
Array.Reverse(arr);
```

⭐ 10. Indices and Ranges (C# 8+)

Index operator (^)

`^1` = last element, `^2` = second last

csharp Copy code

```csharp
int[] a = {10,20,30,40};
int last = a[^1];     // 40
int secondLast = a[^2];   // 30
```

Range operator (start..end)

Creates a sub-array.

csharp Copy code

```csharp
var slice = a[1..3];  // {20, 30}
var fromStart = a[..2]; // {10, 20}
var tillEnd = a[2..];  // {30, 40}
```

⭐ 11. Indexers

Indexers allow objects to be accessed like arrays.

csharp Copy code

```csharp
class MyList
{
    private int[] data = new int[10];

    public int this[int index]
    {
        get { return data[index]; }
        set { data[index] = value; }
    }
}
```

Usage:

csharp

Copy code

```csharp
MyList list = new MyList();
list[0] = 10;          // calls setter
int x = list[0];       // calls getter
```

- ✔ Makes custom collections behave like built-in arrays
- ✔ Indexer can be overloaded by type (e.g., string index)

⭐ Summary Table

Topic	Key Idea
Value types	Stored in stack, copied by value
Reference types	Stored in heap, copied by reference
struct	Lightweight value type
enum	Fixed set of named constants
out	Must be assigned inside method
ref	Must be initialized before passing
nullable types	Allows value types to hold null
nullable reference types	Compiler warning system
??	Null fallback
??=	Assign if null
Arrays	Single, multi-dimensional, jagged
Indexers	Class accessed like array
Ranges	Easy slicing using `..`
Index	Reverse indexing using `^`

If you want, I can prepare next topics:

👉 **Delegates & Events**
👉 **Exception Handling**
👉 **Generics**
👉 **Collections (List, Dictionary, Stack, Queue)**

Just tell me!

Generic classes
Generic methods
Generic Constraints
Collections– generic and non-generic
Collection Examples based on ICollection, IList, IDictionary (both generic and non-generic)
Iterating collections using foreach
Using Tuples to pass multiple values to a function

Here are **clean, structured, exam-oriented notes** on **Generics, Collections, Constraints, Tuples, and Iteration** — perfect for interviews, Viva, and .NET subject exams.

⭐ 1. Generic Classes

A **generic class** allows you to create a class with a **placeholder type**, making it reusable and type-safe.

Example

csharp Copy code

```
class Box<T>
{
    public T Value;
```

```
    public Box(T value)
    {
        Value = value;
    }
}
```

Usage:

csharp Copy code

```
Box<int> b1 = new Box<int>(10);
Box<string> b2 = new Box<string>("Hello");
```

- ✔ No boxing/unboxing
- ✔ Compile-time type safety
- ✔ Code reusability

⭐ 2. Generic Methods

A method that defines its own type parameter.

csharp Copy code

```
class Demo
{
    public void Show<T>(T value)
    {
        Console.WriteLine(value);
    }
}
```

Call:

csharp Copy code

```
Demo d = new Demo();
d.Show<int>(100);
d.Show("Hello");
```

- ✔ Can be used without making whole class generic
- ✔ Type inferred automatically

⭐ 3. Generic Constraints (where)

Constraints restrict what types can be used as generic parameters.

Available Constraints

Constraint	Meaning
`where T : struct`	T must be a value type
`where T : class`	T must be a reference type
`where T : new()`	Must have public parameterless constructor
`where T : BaseClass`	Must inherit BaseClass
`where T : Interface`	Must implement Interface
`where T : unmanaged`	Must be primitive unmanaged type

Example:

csharp Copy code

```csharp
class Repository<T> where T : class, new()
{
    T CreateInstance() => new T();
}
```

- ✔ Multiple constraints allowed
- ✔ `new()` must come last

⭐ 4. Collections – Generic vs Non-Generic

Non-Generic Collections (Old)

Located in: **System.Collections**

Collection	Description
ArrayList	Dynamic array of objects (boxing/unboxing)
Hashtable	Key–value pairs
Queue	FIFO collection
Stack	LIFO collection

❌ Not type-safe
❌ Uses object → requires casting
❌ Slow due to boxing/unboxing

Generic Collections (Recommended)

Located in: **System.Collections.Generic**

Collection	Description
List<T>	Dynamic array
Dictionary<TKey, TValue>	Key–value pairs
Queue<T>	FIFO
Stack<T>	LIFO
HashSet<T>	Unique values
SortedList<TKey,TValue>	Sorted key–value

✔ Type-safe
✔ Faster
✔ No boxing

⭐ 5. Interfaces: ICollection, IList, IDictionary

ICollection (Base Interface)

Represents any collection.

Properties / Methods:

- Count
- Add()
- Remove()
- Clear()
- Contains()

Example (Non-Generic):

csharp Copy code

```csharp
ICollection list = new ArrayList();
list.Add(10);
list.Add("Hello");
```

Example (Generic):

csharp Copy code

```csharp
ICollection<int> nums = new List<int>();
nums.Add(10);
nums.Add(20);
```

IList

Indexed collection (like array).

Non-Generic

csharp Copy code

```csharp
IList list = new ArrayList();
list.Add(1);
list[0] = 10;
```

Generic

csharp Copy code

```csharp
IList<int> l = new List<int>();
l.Add(100);
```

IDictionary

Represents key–value collection.

Non-Generic

csharp Copy code

```csharp
IDictionary table = new Hashtable();
table["id"] = 1;
```

Generic

csharp Copy code

```csharp
IDictionary<int, string> dict = new Dictionary<int, string>();
dict[1] = "One";
```

⭐ 6. Iterating Collections using foreach

`foreach` works on any class implementing `IEnumerable`.

Example

csharp Copy code

```csharp
List<string> names = new List<string>() { "Raj", "Yuvraj", "Amit" };

foreach (var n in names)
{
```

```
    Console.WriteLine(n);
}
```

Works with:

- ✔ Arrays
- ✔ Lists
- ✔ Dictionaries
- ✔ HashSets
- ✔ Custom classes (if implementing IEnumerable)

Dictionary foreach:

csharp Copy code

```
foreach(var kv in dict)
{
    Console.WriteLine($"{kv.Key} : {kv.Value}");
}
```

⭐ 7. Using Tuples to return multiple values

Older Syntax: Tuple Class

csharp Copy code

```
Tuple<int, string> t = Tuple.Create(1, "Raj");
```

⭐ Modern C# Tuples (Recommended)

Returning multiple values:

csharp Copy code

```
(int sum, int product) GetData(int a, int b)
{
```

```
    return (a + b, a * b);
}
```

Usage:

csharp Copy code

```
var result = GetData(10, 20);
Console.WriteLine(result.sum);
Console.WriteLine(result.product);
```

- ✔ No need for class
- ✔ Named tuple fields
- ✔ Lightweight
- ✔ No performance overhead

⭐ Summary Table

Concept	Key Takeaway
Generic Classes	Reusable, type-safe class templates
Generic Methods	Methods with type parameters
Constraints	Restrict valid types using `where`
Generic Collections	Faster, safer (List<T>, Dictionary<T>)
Non-Generic Collections	Older, slower (ArrayList, Hashtable)
ICollection	Base interface for collections
IList	Indexed list
IDictionary	Key–value collection
foreach	Works with any IEnumerable
Tuple	Return multiple values easily

If you want, I can continue with:

👉 **Delegates & Events**
👉 **Exception Handling**
👉 **LINQ**
👉 **Async–Await**

Just tell me!

Delegates
□ Calling methods using delegates
□ Usesofdelegates
□ Multicast delegates
□ Action,Func, Predicate delegates

Here are **clear, detailed, exam-oriented notes** on **Delegates in C#**, covering all the points you listed with examples and real understanding.

⭐ 1. What Is a Delegate?

A **delegate** is a *type-safe function pointer* in C#.
It stores the **reference of a method**, just like variables store values.

✔ Can store **method address**
✔ Can call methods **at runtime**
✔ Used for callbacks, events, LINQ, async programming

Syntax:

csharp Copy code

```
delegate returnType DelegateName(parameterList);
```

⭐ 2. Calling Methods Using Delegates

Step 1: Declare a delegate

csharp Copy code

```csharp
delegate void MyDelegate(string msg);
```

Step 2: Create methods to match signature

csharp Copy code

```csharp
void Show(string message) => Console.WriteLine(message);
```

Step 3: Use delegate

csharp Copy code

```csharp
MyDelegate del = Show;    // assigning method
del("Hello World");       // calling method through delegate
```

✔ Delegate ensures the method signature matches exactly.

⭐ 3. Uses of Delegates

Delegates are used in:

Use Case	Why
Event handling	Forms, WPF, ASP.NET
Callbacks	Asynchronous operations
LINQ	Lambda expressions
Custom sorting	Comparer delegate
Loose coupling	Separate logic from caller

Use Case	Why
Plug-in architecture	Runtime method selection

⭐ 4. Multicast Delegates

A delegate that can hold **multiple methods**.

- ✔ Uses + and – operators
- ✔ Methods are invoked **in sequence**
- ✔ Only the **last method's return value** is used (if return type exists)

Example:

csharp Copy code

```csharp
delegate void MyDel();

void One() { Console.WriteLine("One"); }
void Two() { Console.WriteLine("Two"); }

MyDel d = One;
d += Two;

d();   // prints: One  Two
```

- ✔ Very useful in events, logging, notifications.

⭐ 5. Action, Func, Predicate Delegates

These are **built-in generic delegates** that simplify the use of delegates.

⭐ A. Action Delegate

Represents a method **that returns void**.

✔ 0 to 16 parameters

✔ Always returns **void**

Example:

csharp Copy code

```
Action<string> print = msg => Console.WriteLine(msg);
print("Hello Action");
```

Another example without parameters:

csharp Copy code

```
Action show = () => Console.WriteLine("Welcome");
show();
```

⭐ B. Func Delegate

Represents a method **that returns a value**.

✔ Last type parameter = **return type**

✔ Max 16 parameters

Example:

csharp Copy code

```
Func<int, int, int> add = (a, b) => a + b;
Console.WriteLine(add(10, 20));
```

Meaning:

`Func<parameterType1, parameterType2, returnType>`

⭐ C. Predicate Delegate

Represents a method that **returns bool**.

✔ Always returns **bool**

✔ Takes **one parameter**

Example:

csharp Copy code

```csharp
Predicate<int> isEven = x => x % 2 == 0;

Console.WriteLine(isEven(10));   // True
Console.WriteLine(isEven(11));   // False
```

Commonly used in:

- List<T>.Find()
- RemoveAll()
- Array.FindAll()

⭐ Summary Table

Type	Use	Return Type	Parameters
Delegate	Custom delegate	Any	Any
Action	Built-in	void	0–16
Func	Built-in	anything	0–16, last = return type
Predicate	Built-in	bool	1

⭐ Small Real-Life Example (Delegate + Multicast + Action)

csharp Copy code

```csharp
delegate void Notify();

class Process
```

```
{
    public void Start() => Console.WriteLine("Started");
    public void End() => Console.WriteLine("Ended");
}

var p = new Process();

Notify n = p.Start;
n += p.End;

n();   // Multicast behavior
```

⭐ If you want, I can explain next:

👉 **Events** (built on top of delegates)
👉 **Anonymous Methods & Lambda Expressions**
👉 **Real-life project use of delegates**

Just tell me!

1/2

Anonymous methods
Lambdas

Here are **clean, exam-oriented, interview-friendly notes** on **Delegates, Multicast Delegates, Action/Func/Predicate, Anonymous Methods, and Lambda Expressions** — all explained with simple examples.

⭐ 1. Delegates

A **delegate** is a **type-safe function pointer**.
It stores a reference to a method and allows calling that method indirectly.

Syntax:

csharp Copy code

```csharp
delegate int MyDelegate(int x, int y);
```

Delegate can point to any method with the same **signature** and **return type**.

⭐ 2. Calling Methods Using Delegates

Step 1: Declare delegate

csharp Copy code

```csharp
delegate void ShowMessage(string msg);
```

Step 2: Create method

csharp Copy code

```csharp
void Display(string m)
{
    Console.WriteLine(m);
}
```

Step 3: Use delegate

csharp Copy code

```csharp
ShowMessage d = Display;   // method reference
d("Hello from delegate");  // calling method
```

- ✔ Helps achieve **loose coupling**
- ✔ Used in event-driven programming

⭐ 3. Uses of Delegates

- ✔ Callbacks
- ✔ Event handling (WinForms, WPF, ASP.NET)
- ✔ Plug-in architectures
- ✔ Passing methods as parameters
- ✔ LINQ operations
- ✔ Encapsulation of behavior

Delegate = **method as a variable**

⭐ 4. Multicast Delegates

A delegate that can store **multiple methods** in its invocation list.

Example:

csharp Copy code

```csharp
delegate void MyDel();

void A() => Console.WriteLine("A");
void B() => Console.WriteLine("B");

MyDel d = A;
d += B;          // multicast
d();             // A then B
```

- ✔ Works only when return type is **void**
- ✔ Methods executed in sequence

⭐ 5. Action, Func, Predicate Delegates

These are **predefined delegates** (generic) in .NET.

⭐ Action Delegate

Represents a method that **returns void**.

Examples:

csharp Copy code

```csharp
Action a = () => Console.WriteLine("Hello");
Action<int> print = x => Console.WriteLine(x);
```

- ✔ Can have **0 to 16 input parameters**
- ✔ Always returns void

⭐ Func Delegate

Represents a method that **returns a value**.

Syntax:

csharp Copy code

```csharp
Func<int, int, int> add = (a, b) => a + b;
```

Last type = **return type**, rest = parameters.

Example:

csharp Copy code

```csharp
Func<string> f = () => "Raj";
Func<int, int> square = x => x * x;
```

⭐ Predicate Delegate

Represents a method that **returns a bool** and takes **one input parameter**.

Example:

csharp Copy code

```csharp
Predicate<int> isEven = x => x % 2 == 0;
```

```
Console.WriteLine(isEven(10));   // True
```

Used heavily in:

- ✔ List<T>.Find
- ✔ List<T>.RemoveAll
- ✔ validation logic

⭐ 6. Anonymous Methods

A method **without a name**, created using the `delegate` keyword.

Example:

csharp Copy code

```
delegate void MyDel(string msg);

MyDel d = delegate(string s)
{
    Console.WriteLine(s);
};

d("Hello Anonymous Method");
```

- ✔ Can access outer variables (closure)
- ✔ Useful for short temporary logic
- ✔ Used before lambda expressions became popular

⭐ 7. Lambda Expressions

Short, modern syntax for writing methods inline.

Syntax:

csharp Copy code

```
(parameters) => expression
```

Examples:

1. No parameters

csharp Copy code

```csharp
Action a = () => Console.WriteLine("Hello");
```

2. One parameter

csharp Copy code

```csharp
Action<int> print = x => Console.WriteLine(x);
```

3. Multiple parameters

csharp Copy code

```csharp
Func<int, int, int> add = (a, b) => a + b;
```

4. Block body

csharp Copy code

```csharp
Func<int, bool> check = x =>
{
    return x > 0;
};
```

- ✔ Most used with LINQ
- ✔ Cleaner, shorter, faster to write
- ✔ Replaces anonymous methods

⭐ 8. Differences Between Delegate, Anonymous Method, and Lambda

Feature	Delegate	Anonymous Method	Lambda Expression
Syntax	Long	Medium	Short & clean

Feature	Delegate	Anonymous Method	Lambda Expression
Needs method name	Yes	No	No
Closure support	Yes	Yes	Yes
LINQ usage	Rare	Rare	Most common
Modern code	Less preferred	Less preferred	Most preferred

⭐ Final Summary

- ✔ **Delegate** → method pointer
- ✔ **Multicast delegate** → multiple methods
- ✔ **Action** → returns void
- ✔ **Func** → returns value
- ✔ **Predicate** → returns bool
- ✔ **Anonymous method** → unnamed method using `delegate`
- ✔ **Lambda expression** → shortest syntax for inline methods

---

If you want, I can continue with:

- 👉 **Events**
- 👉 **Exception Handling**
- 👉 **LINQ basics + queries**
- 👉 **Async & Await**

Just tell me!

Error Handling (Exceptions Handling)
 Checked&UncheckedStatements

 Thetry,catch, finally
 Dos&Don'tsofExceptionHandling

Below are **clean, structured, exam-oriented and interview-friendly notes** on:

- ✔ Delegates
- ✔ Multicast delegates
- ✔ Action / Func / Predicate
- ✔ Anonymous methods
- ✔ Lambdas
- ✔ Exception handling
- ✔ Checked/Unchecked

All covered in an easy-to-understand format with examples.

⭐ 1. Delegates in C#

A **delegate** is a type-safe function pointer.
It holds the reference of a method and calls the method at runtime.

Syntax

csharp Copy code

```csharp
delegate void MyDelegate(string msg);
```

Assigning and Calling

csharp Copy code

```csharp
MyDelegate d = Print;
d("Hello");

void Print(string s)
{
    Console.WriteLine(s);
}
```

- ✔ Ensures **type safety**
- ✔ Enables **callbacks**
- ✔ Supports **multicast**

⭐ 2. Calling Methods Using Delegates

Delegates act as objects:

csharp Copy code

```csharp
public delegate int Operation(int x, int y);

static int Add(int a, int b) => a + b;

Operation op = Add;

int result = op(10, 20);
```

⭐ 3. Uses of Delegates

Use-case	Explanation
Event Handling	Events in .NET are built using delegates
Callbacks	Function passed into another function
Asynchronous programming	Background operations
Strategy pattern	Passing different algorithms
LINQ	Uses lambda and delegates

⭐ 4. Multicast Delegates

A delegate can hold **multiple method references** using `+=` .

Example

csharp Copy code

```csharp
delegate void Show();

void A() => Console.WriteLine("A");
void B() => Console.WriteLine("B");

Show s = A;
s += B;

s();   // Output: A  B
```

- ✔ Only works for **void return type**
- ✔ Methods execute in order

⭐ 5. Action, Func, Predicate Delegates

These are **predefined delegates** provided by .NET.

✔ Action

- Returns **void**
- Can take **0 to 16 parameters**

csharp Copy code

```csharp
Action<string> log = msg => Console.WriteLine(msg);
log("Hello");
```

✔ Func

- Returns **a value**
- Last parameter is the **return type**

csharp Copy code

```
Func<int, int, int> add = (x, y) => x + y;
int res = add(2, 3);    // 5
```

✔️ Predicate

- Returns **bool**
- Has exactly **one parameter**

csharp Copy code

```
Predicate<int> isEven = n => n % 2 == 0;
bool result = isEven(10);   // true
```

⭐ 6. Anonymous Methods

Methods **without a name**, used with delegates.

csharp Copy code

```
delegate void MyDel(string msg);

MyDel d = delegate (string m)
{
    Console.WriteLine(m);
};

d("Hello");
```

✔️ Useful when method required only once
✔️ Introduced before lambda expressions

⭐ 7. Lambda Expressions

Short syntax for delegates/anonymous methods.

Syntax:

csharp Copy code

```
(parameters) => expression
```

Example:

csharp Copy code

```
Action<string> show = s => Console.WriteLine(s);
show("Hi");
```

Lambda with body:

csharp Copy code

```
Func<int, int> square = n =>
{
    int result = n * n;
    return result;
};
```

Used heavily in:

- ✔ Delegates
- ✔ LINQ
- ✔ Events
- ✔ Async programming

⭐ 8. Exception Handling in C#

Used to **handle runtime errors** gracefully.

⭐ try-catch-finally

csharp Copy code

```
try
{
```

```
    int x = 10 / 0;
}
catch (DivideByZeroException ex)
{
    Console.WriteLine("Cannot divide by zero");
}
finally
{
    Console.WriteLine("Always runs");
}
```

✔ try

Code that may throw an exception

✔ catch

Handle specific exceptions

✔ finally

Always executes (close DB connections, files, etc.)

⭐ 9. Checked & Unchecked Statements

Used to control **overflow checking** for numeric operations.

✔ checked (throws exception)

csharp Copy code

```
checked
{
    int x = int.MaxValue;
    x++;       // throws OverflowException
}
```

✔ unchecked (ignores overflow)

csharp Copy code

```
unchecked
{
    int x = int.MaxValue;
    x++;       // wraps around (no error)
}
```

⭐ 10. Dos and Don'ts of Exception Handling

✔️ Do:

- Catch **specific exceptions**
- Use `finally` to clean up resources
- Throw custom exceptions when needed
- Use `using` for automatic disposal
- Log the errors

❌ Don't:

- Don't catch `Exception` unless necessary
- Don't swallow exceptions silently
- Don't use exceptions for normal flow
- Don't expose internal exceptions (security risk)
- Don't put large code in try block

⭐ Quick Summary Table

Concept	Key Idea
Delegate	Type-safe function pointer
Multicast delegate	Multiple methods in one delegate
Action	void method with parameters
Func	Method returning a value
Predicate	Method returning bool

Concept	Key Idea
Anonymous method	delegate + inline method
Lambda	Compact syntax for delegates
try/catch/finally	Exception handling structure
checked	Overflow checking enabled
unchecked	Overflow ignored

If you want, I can continue with:

👉 **Events**
👉 **Async / Await**
👉 **LINQ Queries**
👉 **Threading and Task Parallelism (TPL)**

Just tell me!

User Defined Exception classes
Declaring and raising events
Handling events

Below are **clean, structured, interview-ready notes** for all the remaining topics:

✔ Delegates
✔ Anonymous Methods & Lambdas
✔ Exception Handling
✔ User-Defined Exceptions
✔ Events: Declaring & Handling

Each topic is precise, with examples and key points.

⭐ 1. Delegates in C#

A **delegate** is a type-safe function pointer used to store references to methods.

Syntax

csharp Copy code

```csharp
public delegate void MyDelegate(string msg);
```

Assigning & Calling

csharp Copy code

```csharp
void Show(string s) => Console.WriteLine(s);

MyDelegate del = Show;
del("Hello");
```

- ✔ Delegate stores method reference
- ✔ Invoked like a method
- ✔ Ensures type safety

⭐ 2. Calling Methods Using Delegates

Example

csharp Copy code

```csharp
int Add(int a, int b) => a + b;

delegate int MathOp(int a, int b);

MathOp op = Add;
int result = op(10, 20);
```

⭐ 3. Uses of Delegates

- ✔ Event handling
- ✔ Callback functions
- ✔ Passing methods as parameters
- ✔ LINQ expressions
- ✔ Asynchronous programming

⭐ 4. Multicast Delegates

A multicast delegate can store **multiple method references**.

csharp Copy code

```csharp
delegate void MyDel();

void A() => Console.WriteLine("A");
void B() => Console.WriteLine("B");

MyDel d = A;
d += B;

d(); // Calls A then B
```

- ✔ Only possible for delegates returning `void`

⭐ 5. Inbuilt Delegates: Action, Func, Predicate

Action

- No return value
- 0 to 16 parameters

csharp Copy code

```csharp
Action<string> print = Console.WriteLine;
print("Hello");
```

Func

- Returns a value
- Last type parameter is return type

csharp Copy code

```csharp
Func<int, int, int> add = (a, b) => a + b;
Console.WriteLine(add(10, 20));
```

Predicate

- Returns **bool**
- Takes **only one parameter**

csharp Copy code

```csharp
Predicate<int> isEven = x => x % 2 == 0;
Console.WriteLine(isEven(10));
```

⭐ 6. Anonymous Methods

Anonymous methods are methods **without a name**, created using the `delegate` keyword.

csharp Copy code

```csharp
Action a = delegate()
{
    Console.WriteLine("Inside anonymous method");
};
```

✔ Useful for short, inline logic
✔ Precursor to lambda expressions

⭐ 7. Lambda Expressions

Cleaner, shorter syntax for functions.

Syntax

csharp Copy code

```
(parameters) => expression;
```

Example:

csharp Copy code

```
Func<int, int> square = x => x * x;
Console.WriteLine(square(5));
```

Lambda with block:

csharp Copy code

```
Action greet = () =>
{
    Console.WriteLine("Hello!");
};
```

- ✔ Used heavily in LINQ
- ✔ Short, clean, functional style

⭐ 8. Exception Handling

Exception: runtime error handled by **try–catch–finally**.

---

checked & unchecked

checked

Enables overflow checking.

csharp Copy code

```
checked
{
    int x = int.MaxValue + 1; // throws OverflowException
}
```

unchecked

Ignores overflow.

csharp Copy code

```
unchecked
{
    int x = int.MaxValue + 1; // wraps around silently
}
```

⭐ try, catch, finally

csharp Copy code

```
try
{
    int a = 10 / 0;
}
catch (DivideByZeroException ex)
{
    Console.WriteLine(ex.Message);
}
finally
{
    Console.WriteLine("Always executes");
}
```

- ✔ `finally` always executes
- ✔ Use specific exceptions, not generic ones

⭐ Dos & Don'ts of Exception Handling

✔ DO

- Catch **specific** exceptions
- Use `finally` to release resources
- Throw meaningful messages
- Use custom exceptions where needed
- Log exceptions

❌ DON'T

- Never write `catch(Exception) {}` (bad practice)
- Don't use exceptions for flow control
- Don't swallow exceptions
- Avoid throwing exceptions inside `finally`
- Don't rethrow using `throw ex;` → loses stack trace
  - Use `throw;` instead

⭐ User-Defined Exception Classes

Must derive from **Exception** class.

Example:

csharp Copy code

```csharp
class AgeException : Exception
{
    public AgeException(string msg) : base(msg) { }
}
```

Throwing it:

csharp Copy code

```csharp
void CheckAge(int age)
{
    if (age < 18)
```

```
        throw new AgeException("Age must be 18+");
}
```

⭐ Declaring Events

Events are built on delegates.

Step 1: Declare Delegate

csharp Copy code

```csharp
public delegate void Notify();
```

Step 2: Declare Event

csharp Copy code

```csharp
public event Notify ProcessCompleted;
```
